

# CREDALJT: Exact Marginal Inference in Logical Credal Networks at Treewidth Cost

Algorithm, a fully worked example, and the exactness theory

IBM Research

## Abstract

Marginal inference in a Logical Credal Network (LCN) is the problem of computing tight lower and upper bounds on the probability of a query atom over the credal set  $\mathcal{D}_{\text{LCN}}$  that the program's sentences and independence assumptions induce. The direct formulation optimizes over the full  $2^n$  joint and does not scale; the credal-network engines scale but optimize over a strictly larger set, the strong extension  $\mathcal{D}_{\text{SE}}$ , and are therefore loose in ways that are hard to predict. This note presents CREDALJT (scheme D5), which occupies neither position: it targets  $\mathcal{D}_{\text{LCN}}$  *directly*, yet its cost is exponential in the treewidth  $w$  rather than in  $n$ .

CREDALJT builds a single junction tree over the chain-graph moral graph, assigns each cluster a variable vector over the joint of its atoms, ties adjacent clusters by separator-consistency equalities, hosts every sentence and every non-redundant independence equality on a cluster containing its scope, and optimizes the query read-off with a certified global solver. We give the algorithm in full (Section 3), write the resulting nonlinear program formally (Section 4), work a complete four-atom example end to end including every one of its 22 variables and 22 constraints (Section 5), and consolidate the exactness theory with complete proofs (Section 6): a repaired running-intersection reconstruction lemma, the exactness theorem under hypotheses H1–H4, the unconditional outer-bound guarantee that survives when H2 and H3 fail, and a corollary giving exactness on the **ktree-fr** benchmark family at  $O(n2^{k+1})$ . Section 7 is a self-contained subsection ready for inclusion in a paper. Every numerical value reported here was computed on the stated instance and independently cross-checked (Section 9).

## Contents

|          |                                                           |          |
|----------|-----------------------------------------------------------|----------|
| <b>1</b> | <b>Introduction</b>                                       | <b>2</b> |
| <b>2</b> | <b>Preliminaries</b>                                      | <b>3</b> |
| 2.1      | The credal set $\mathcal{D}_{\text{LCN}}$ . . . . .       | 3        |
| 2.2      | Chain graph, strong extension, and the two gaps . . . . . | 4        |
| <b>3</b> | <b>The CredalJT algorithm</b>                             | <b>5</b> |
| 3.1      | Overview . . . . .                                        | 5        |
| 3.2      | Constructing the cluster tree . . . . .                   | 5        |
| 3.3      | Hosting and the assertion triage . . . . .                | 5        |
| 3.4      | Solving . . . . .                                         | 7        |
| 3.5      | Complexity . . . . .                                      | 7        |
| <b>4</b> | <b>The nonlinear program</b>                              | <b>8</b> |

|          |                                                         |           |
|----------|---------------------------------------------------------|-----------|
| <b>5</b> | <b>A worked example</b>                                 | <b>9</b>  |
| 5.1      | The instance . . . . .                                  | 10        |
| 5.2      | The junction tree . . . . .                             | 10        |
| 5.3      | Separator-entailment discharges the assertion . . . . . | 11        |
| 5.4      | The program in full . . . . .                           | 11        |
| 5.5      | Results . . . . .                                       | 12        |
| 5.6      | Two contrasting structures . . . . .                    | 12        |
| <b>6</b> | <b>Exactness</b>                                        | <b>12</b> |
| 6.1      | Position among the engines . . . . .                    | 16        |
| <b>7</b> | <b>A self-contained subsection for a paper</b>          | <b>17</b> |
| <b>8</b> | <b>Experimental setup</b>                               | <b>18</b> |
| <b>9</b> | <b>Provenance</b>                                       | <b>18</b> |

## 1 Introduction

A Logical Credal Network specifies imprecise probabilistic knowledge by bounding the probabilities of propositional formulas: a *sentence* is either  $\ell \leq P(\varphi) \leq u$  (Type-1) or  $\ell \leq P(\varphi \mid \psi) \leq u$  (Type-2), over propositional formulas  $\varphi, \psi$  on binary atoms. Together with the independence assumptions that the Local Markov Condition derives from the program’s structure, a set of sentences determines a closed set  $\mathcal{D}_{\text{LCN}}$  of joint distributions, and the inference task is to compute  $\underline{P}(a) = \min_{p \in \mathcal{D}_{\text{LCN}}} P_p(a)$  and  $\overline{P}(a) = \max_{p \in \mathcal{D}_{\text{LCN}}} P_p(a)$  for each atom  $a$ .

Two families of engines exist. The *direct* formulation writes one variable per joint state and optimizes over  $\mathcal{D}_{\text{LCN}}$  itself; it is exact when the solver certifies global optimality, but its  $2^n$  size confines it to small instances, and even there the certificate sometimes fails to materialize (Remark 3). The *credal-network* family (Credal VE, Credal CTE, ApproxLP, Interval BP, Credal IJGP) compiles the program into a chain-graph credal network with interval local credal sets, and propagates; it scales, but it optimizes over the strong extension  $\mathcal{D}_{\text{SE}} \supseteq \mathcal{D}_{\text{LCN}}$  and so inherits two distinct looseness gaps (Section 2.2).

CREDALJT belongs to neither family. It does not compile a credal network: it never reads the per-family interval bounds and never enumerates extreme points, consuming only the chain graph’s *structure* and the LCN’s own sentences and independence assertions. Consequently it optimizes over a relaxation of  $\mathcal{D}_{\text{LCN}}$  built from cluster marginals, not over  $\mathcal{D}_{\text{SE}}$  — and under the hypotheses of Theorem 1 that relaxation is exact. Its cost is  $\sum_C 2^{|C|} = O(n 2^{w+1})$ , exponential in treewidth rather than in  $n$ .

### Contributions.

1. A complete specification of the algorithm (Section 3, Algorithm 1), including the constraint-hosting triage that keeps the treewidth small and the query-independence property that lets one constraint program serve every atom.
2. The nonlinear program written formally (Section 4), with the marginalization operator, the cleared Type-2 encoding, and the joint (non-decomposed) form of the independence equalities made explicit.

3. A four-atom example worked end to end (Section 5): the junction tree, all 22 decision variables, all 22 constraints transcribed individually, the resulting bounds, and an independent cross-check against a certified global reference on every atom.
4. The exactness theory consolidated with complete proofs (Section 6): Lemma 2 (running-intersection reconstruction, stated correctly in the presence of zero separator cells), Theorem 1 (exactness under H1–H4, and the unconditional outer-bound half), Remark 6 (a genuine soundness caveat for conditional queries), and Corollary 1 (**ktree-fr** exactness at  $O(n2^{k+1})$ ).
5. A one-page self-contained subsection for direct inclusion in a paper (Section 7).

## 2 Preliminaries

Throughout,  $\mathbf{V}$  is a set of  $n$  binary atoms,  $\Delta(2^{\mathbf{V}})$  the simplex of joint distributions over the  $2^n$  truth assignments, and  $p \upharpoonright A$  the marginal of  $p$  on an atom subset  $A \subseteq \mathbf{V}$ . For a propositional formula  $\varphi$  we write  $\text{atoms}(\varphi)$  for its atoms and  $\omega \models \varphi$  for satisfaction.

### 2.1 The credal set $\mathcal{D}_{\text{LCN}}$

A Type-1 sentence  $\ell \leq P(\varphi) \leq u$  contributes the two linear rows  $\ell \leq \sum_{\omega \models \varphi} p(\omega) \leq u$ . A Type-2 sentence  $\ell \leq P(\varphi \mid \psi) \leq u$  is read in *cleared* form,

$$\ell \cdot P(\psi) \leq P(\varphi \wedge \psi) \leq u \cdot P(\psi), \quad (1)$$

which avoids a division and is the form imposed throughout.

**Remark 1** (The cleared reading is the one in force). The cleared rows (1) are vacuously satisfied on  $\{p : P(\psi) = 0\}$ , so they admit joints that the ratio reading  $\ell \leq P(\varphi \wedge \psi)/P(\psi) \leq u$  excludes. The two readings therefore define different credal sets. Every statement in this note is made for the cleared reading.

The Local Markov Condition attaches to the program a set of conditional independence assertions  $(X \perp\!\!\!\perp Y \mid S)$  with  $X$  a singleton. Each is imposed as the family of *joint* equalities

$$P(\mathbf{x}, \mathbf{y}, \mathbf{s}) P(\mathbf{s}) = P(\mathbf{x}, \mathbf{s}) P(\mathbf{y}, \mathbf{s}), \quad (2)$$

one for each configuration  $(\mathbf{y}, \mathbf{s})$  of  $Y \cup S$ . Writing  $g_s$  for the functional of a sentence  $s$  and  $h_\iota$  for the residual of an assertion  $\iota$ ,

$$\mathcal{D}_{\text{LCN}} = \{ p \in \Delta(2^{\mathbf{V}}) : \ell_s \leq g_s(p) \leq u_s \ \forall s, \quad h_\iota(p) = 0 \ \forall \iota \}, \quad (3)$$

and the inference targets are  $\underline{P}(a) = \min_{p \in \mathcal{D}_{\text{LCN}}} P(a)$ ,  $\overline{P}(a) = \max_{p \in \mathcal{D}_{\text{LCN}}} P(a)$ .

The following elementary lemma is the workhorse of Section 6: it is what licenses imposing a constraint on a single cluster.

**Lemma 1** (Locality). *Every constraint in (3) depends on  $p$  only through its marginal on a fixed atom set, its scope: a sentence  $s$  through  $p \upharpoonright \text{atoms}(s)$ , and an LMC assertion  $(X \perp\!\!\!\perp Y \mid S)$  through  $p \upharpoonright (X \cup Y \cup S)$ . Consequently, if  $p$  and  $p'$  satisfy  $p \upharpoonright A = p' \upharpoonright A$  for the scope  $A$  of a constraint  $c$ , then  $c$  holds for  $p$  iff it holds for  $p'$ .*

*Proof.* Let  $c$  be a constraint with scope  $A$ . There are three cases.

*Type-1.*  $P(\varphi) = \sum_{\omega \models \varphi} p(\omega)$ . Since  $\text{atoms}(\varphi) \subseteq A$ , the truth value of  $\varphi$  at  $\omega$  is determined by  $\omega \upharpoonright A$ ; grouping the sum by the value of  $\omega \upharpoonright A$  gives  $P(\varphi) = \sum_{\alpha \models \varphi} (p \upharpoonright A)(\alpha)$ , where  $\alpha$  ranges over assignments of  $A$ . This is a function of  $p \upharpoonright A$  alone.

*Type-2.* By the previous case both  $P(\varphi \wedge \psi)$  and  $P(\psi)$  are functions of  $p \upharpoonright A$  with  $A \supseteq \text{atoms}(\varphi) \cup \text{atoms}(\psi)$ ; the rows (1) are therefore functions of  $p \upharpoonright A$ .

*LMC.* Each factor in (2) is a probability of a conjunction of literals over  $X \cup Y \cup S$ , hence by the Type-1 case a function of  $p \upharpoonright (X \cup Y \cup S)$ ; a product of such functions is again one.

In each case the constraint's truth value is determined by  $p \upharpoonright A$ , which is precisely the stated consequence.  $\square$

## 2.2 Chain graph, strong extension, and the two gaps

The LCN pipeline moralizes and simplifies the program's structure graph into a *chain graph* whose nodes are chain components; an undirected component collapses into a single *compound* node (rendered with a --joined name, e.g. C-D). This yields a factorization

$$P(\mathbf{V}) = \prod_v P(\text{ch}_v \mid \text{pa}_v), \quad (4)$$

one factor per node  $v$ , where  $\text{ch}_v$  is the (possibly multi-atom) chain component at  $v$  and  $\text{pa}_v$  its parents. The *family scope* at  $v$  is  $\text{ch}_v \cup \text{pa}_v$ ; root families have  $\text{pa}_v = \emptyset$ .

The credal-network engines replace each factor by an interval local credal set  $K_{v,\pi}$  per parent configuration  $\pi$  and propagate over the *strong extension*

$$\mathcal{D}_{\text{SE}} = \left\{ \prod_v P_v : P_v \in K_{v,\pi} \text{ chosen independently for each parent configuration } \pi \right\} \supseteq \mathcal{D}_{\text{LCN}}. \quad (5)$$

The inclusion is generally strict, for two reasons that it is worth naming since they are exactly what CREDALJT avoids:

- **Gap A (local-set widening).** Each  $K_v$  is computed per family in isolation, so it may admit conditionals that no global  $p \in \mathcal{D}_{\text{LCN}}$  realizes: it ignores sentences on *other* families and the cross-family LMC equalities.
- **Gap B (strong-extension widening).** Even with exact local sets, the free product in (5) lets each family pick its row independently, which can violate a sentence whose scope spans several families.

CREDALJT reads neither the interval bounds nor the extreme points of any  $K_{v,\pi}$ ; it uses (4) only for its *scopes*. Neither gap therefore arises, and the target is  $\mathcal{D}_{\text{LCN}}$  itself. This is the structural reason it is the only engine in the credal-network directory whose bound character is stated against  $\mathcal{D}_{\text{LCN}}$  rather than  $\mathcal{D}_{\text{SE}}$ .

**Definition 1** (Bound character). Let  $\mathcal{R}$  be a reference set (here  $\mathcal{D}_{\text{LCN}}$  or  $\mathcal{D}_{\text{SE}}$ ) with true bounds  $[q^*, \bar{q}^*]$  for the query at hand. An engine's interval  $[q, \bar{q}]$  is *exact w.r.t.*  $\mathcal{R}$  if  $[q, \bar{q}] = [q^*, \bar{q}^*]$ ; a valid *outer* bound if  $[q, \bar{q}] \supseteq [q^*, \bar{q}^*]$ ; an *inner* bound if  $[q, \bar{q}] \subseteq [q^*, \bar{q}^*]$ ; and *unreliable* if neither containment is guaranteed. An outer bound is sound as a guarantee; an inner bound is not.

## 3 The CredalJT algorithm

### 3.1 Overview

CREDALJT proceeds in four steps. It builds a cluster tree over the atoms from the family scopes of the chain graph (4); it assigns every sentence and every non-redundant independence assertion to a cluster containing its scope; it writes a single constraint system whose variables are one distribution per cluster; and it reads the query off a cluster containing it, minimizing and maximizing over that system. Algorithm 1 states the procedure; Section 4 writes the constraint system formally.

The only input the algorithm takes from the chain graph is the collection of family *scopes*  $\{\text{ch}_v \cup \text{pa}_v\}_v$ , together with the LCN’s own sentences and assertions. It never forms the local credal sets  $K_{v,\pi}$  of (5), and never enumerates their extreme points. This is the structural reason its target is  $\mathcal{D}_{\text{LCN}}$  rather than  $\mathcal{D}_{\text{SE}}$ : neither Gap A nor Gap B can arise from a construction that does not build the objects in which those gaps live.

Throughout,  $\mathcal{C}$  denotes the clusters,  $A(C) \subseteq \mathbf{V}$  the atoms of cluster  $C$ ,  $S_{CD} = A(C) \cap A(D)$  the separator on a tree edge, and

$$w + 1 = \max_{C \in \mathcal{C}} |A(C)|, \quad (6)$$

so  $w$  is the treewidth of the triangulated graph and  $w + 1$  the largest cluster measured in atoms. Cluster sizes are always reported in atoms below.

### 3.2 Constructing the cluster tree

The scopes to be covered are the family scopes, augmented by one clique per *cross-family sentence* — a sentence whose atoms fit inside no single family scope. Augmenting forces such a sentence’s atoms to co-occur in a cluster, which is what guarantees it a host in Section 3.3.

Independence assertions are deliberately *not* used to augment. They are structural, and a cluster tree with the running-intersection property already enforces them through its separators (Section 3.3); adding them as cliques would inflate the width for no gain. A Markov chain, for instance, would collapse from width 2 to a single cluster containing all  $n$  atoms.

Given the scopes, an elimination order is computed greedily by *min-fill* on the interaction graph — the graph in which two nodes are adjacent when they co-occur in some scope — eliminating at each step a vertex whose elimination adds fewest fill-in edges. Buckets are then formed along that order, each scope placed in the bucket of the first order variable it contains; a bucket’s cluster is the union of its scopes, and its parent is the first later order variable appearing in the cluster after the bucket variable is removed. This is standard bucket elimination, so **the resulting tree has the running-intersection property by construction**, discharging hypothesis H1 of Theorem 1 without further argument.

Each cluster’s atom set  $A(C)$  is the union of the atoms of its nodes; a compound node contributes all atoms of its chain component at once, so a cluster may be wider in atoms than in nodes. Finally, any *leaf* whose atom set is contained in its parent’s is removed, repeatedly: such a leaf is redundant, since the parent already carries that joint, and removing it is safe because a leaf has no children to reattach. Only subsumed leaves are removed, so the tree is not claimed to be minimal — a non-maximal cluster may survive in the interior.

### 3.3 Hosting and the assertion triage

Say a constraint is *hosted* on a cluster  $C$  when  $\text{scope}(c) \subseteq A(C)$ ; by Lemma 1 its value is then a function of  $q_C$  alone. Hosting each constraint on a single cluster suffices: separator consistency

---

**Algorithm 1** CREDALJT: marginal bounds at treewidth cost

---

**Require:** sentences  $\Sigma$ , LMC assertions  $\Lambda$ , family scopes  $\mathcal{F}$ , evidence  $e$ , width budget  $b$

**Ensure:**  $[P(a | e), \bar{P}(a | e)]$  for every atom  $a \notin \text{atoms}(e)$

```
1:  $\mathcal{S} \leftarrow \mathcal{F} \cup \{\text{atoms}(s) : s \in \Sigma \text{ cross-family}\}$  ▷ assertions do not augment
2:  $T \leftarrow \text{BUCKETTREE}(\mathcal{S}, \text{MINFILLORDER}(\mathcal{S}))$ ;  $A(C) \leftarrow \bigcup_{v \in C} \text{atoms}(v)$  ▷ RIP holds [H1]
3: remove subsumed leaves;  $S_{CD} \leftarrow A(C) \cap A(D)$  for each edge
4: for all  $s \in \Sigma$  do ▷ every sentence must be hosted [H2]
5:   host  $s$  on a cluster  $C$  with  $\text{atoms}(s) \subseteq A(C)$ , if one exists
6: end for
7: if  $\max_C |A(C)| > b$  or some  $s \in \Sigma$  was not hosted then
8:   return  $\text{DIRECT}(a, e)$  for every atom  $a \notin \text{atoms}(e)$  ▷  $2^n$ , uncertified; Rem. 3
9: end if
10: for all  $\iota = (X \perp\!\!\!\perp Y \mid S) \in \Lambda$  do ▷ three-way triage [H3]
11:   if  $S$  separates  $X$  from  $Y$  in  $G^*$  then
12:     skip  $\iota$  ▷ entailed by the separators
13:   else if  $X \cup Y \cup S \subseteq A(C)$  for some  $C$  then
14:     host  $\iota$  on  $C$ 
15:   else
16:     drop  $\iota$  ▷ bound stays outer, may be loose
17:   end if
18: end for
19: build  $\mathcal{G}$ :  $q_C \in \Delta(2^{|A(C)|})$ ,  $q_C \upharpoonright S_{CD} = q_D \upharpoonright S_{CD}$ , and (12)–(14) on each host ▷ once
20: for all atoms  $a \notin \text{atoms}(e)$  do
21:   if  $\{a\} \cup \text{atoms}(e) \subseteq A(C_a)$  for some  $C_a$  then ▷ objective only [H4]
22:      $\underline{P} \leftarrow \min_{q \in \mathcal{G}} \rho_a(q)$ ,  $\bar{P} \leftarrow \max_{q \in \mathcal{G}} \rho_a(q)$  ▷ (15), (16)
23:   else
24:      $[\underline{P}, \bar{P}] \leftarrow \text{DIRECT}(a, e)$ 
25:   end if
26:   clamp  $[\underline{P}, \bar{P}]$  to  $[0, 1]$ 
27: end for
```

---

propagates its effect to the rest of the tree, as the reconstruction argument of Theorem 1 makes precise.

Every sentence is hosted if it can be. A sentence with no host cannot be represented exactly on this tree, and the algorithm falls back to the direct formulation (Section 3.4).

Independence assertions admit a three-way triage, and this is the algorithm’s most consequential step:

1. **Separator-entailed  $\Rightarrow$  omitted.** If the conditioning set  $S$  separates  $X$  from  $Y$  in the chordal graph  $G^*$  (Definition 2), every separator-consistent family of cluster marginals already satisfies  $(X \perp\!\!\!\perp Y \mid S)$ , so imposing it would be redundant. This is what keeps a chain at width 2.
2. **Hosted  $\Rightarrow$  imposed** as the bilinear equalities (2) on the host cluster.
3. **Neither  $\Rightarrow$  dropped.**

Case 3 is the algorithm’s only source of looseness, and it is exactly the failure of hypothesis H3. The bound remains valid — the projection half of Theorem 1 requires neither H2 nor H3 — but it can be strictly loose; Remark 5 exhibits an instance where it is.

### 3.4 Solving

The constraint system of Step 3 mentions no query, and every atom lies in some cluster because each family scope is itself a clique. One system therefore serves all marginals: per atom, only the objective changes, and two optimizations are solved. This is the sense in which the construction is *query-independent*.

**Global solution.** The system is nonconvex: the assertion equalities (14) are bilinear, and under evidence so is the ratio equality (16). Exactness in Theorem 1 refers to the global optimum, so the bounds reported here were computed with a spatial branch-and-bound solver at gap tolerance 0.

**Remark 2** (A local solver yields an inner bound). Solved instead by a local method from a finite set of starting points — keeping the smallest minimum and largest maximum found — the procedure returns a value no more extreme than the true optimum in each direction. The resulting interval is therefore contained in the exact one: an *inner* bound in the sense of Definition 1, and not a guarantee. Global solution is what makes Theorem 1 applicable.

**Fallback.** Two conditions divert a query to the direct  $2^n$  formulation: some cluster exceeding the width budget  $b$ , and a sentence or query having no host. The first is global to the instance; the second may be local to one atom, arising when that atom and the evidence share no cluster, i.e. when H4 fails for it alone.

**Remark 3** (The fallback is exact in formulation, not in certificate). The fallback optimizes the true  $2^n$  program over  $\mathcal{D}_{\text{LCN}}$ , but by a local method, so its output carries no global guarantee and can be over-tight. Conversely, on instances where the  $2^n$  formulation is solvable but its global solver fails to certify some endpoint, the cluster program of Section 4 is often the more reliable route to a bound, even at  $n$  small enough for the joint to be written down.

A failed optimization is reported as the corresponding vacuous endpoint (0 for a minimum, 1 for a maximum): uninformative, never unsound. If every marginal returns vacuous, the instance or the evidence is likely inconsistent.

### 3.5 Complexity

*Variables.* One per cluster state, so by (6)

$$\sum_{C \in \mathcal{C}} 2^{|A(C)|} = O(n 2^{w+1}), \quad (7)$$

since  $|\mathcal{C}| = O(n)$ , plus one auxiliary scalar per atom under evidence. Against  $2^n$  for the direct formulation, this is the algorithm’s reason for being: the cost is exponential in the treewidth, not in the number of atoms.

*Constraints.* There are  $|\mathcal{C}|$  normalization equalities;  $\sum_{(C,D), S_{CD} \neq \emptyset} 2^{|S_{CD}|} = O(n 2^w)$  separator equalities; two rows per hosted sentence; and, per hosted assertion  $(X \perp\!\!\!\perp Y \mid S)$ ,

$$2^{|Y|} \cdot \max(1, 2^{|S|}) \quad \text{equalities.} \quad (8)$$

**Remark 4** (Which rows are nonconvex, and why the width bound survives). (a) Only the assertion equalities (14) and, under evidence, the ratio equality (16) are nonconvex. The cleared Type-2 rows (13) are *linear* in  $q$ , since  $\ell$  and  $u$  are constants. Consequently an instance with no evidence and no

hosted assertion yields a *linear* program — which is exactly the situation in Section 5, despite that instance being loopy. (b) The count (8) appears unbounded in  $|Y|$ , but a *hosted* assertion satisfies  $X \cup Y \cup S \subseteq A(C)$ , so it contributes at most  $2^{w+1}$  rows and (7) is unaffected. An assertion with a wide  $Y$  is precisely one that the triage of Section 3.3 either discharges structurally or drops.

Per atom the cost is two optimizations over the shared system, hence  $2|\mathbf{V}|$  in total.

## 4 The nonlinear program

Fix a cluster  $C$  and index the  $2^{|A(C)|}$  assignments of  $A(C)$  by  $i$ , writing  $\omega_i^C$  for the  $i$ -th. For a formula  $\varphi$  with  $\text{atoms}(\varphi) \subseteq A(C)$  let  $A_\varphi^{(C)} \in \{0, 1\}^{2^{|A(C)|}}$  be its indicator vector,  $A_\varphi^{(C)}[i] = \mathbf{1}[\omega_i^C \models \varphi]$ , and abbreviate  $\langle v, q_C \rangle = \sum_i v_i q_{C,i}$ ; thus  $\langle A_\varphi^{(C)}, q_C \rangle = P(\varphi)$  whenever  $q_C$  is the marginal of a joint on  $A(C)$ .

**Variables and normalization.** One distribution per cluster,

$$q_C \in [0, 1]^{2^{|A(C)|}}, \quad \sum_{i=0}^{2^{|A(C)|}-1} q_{C,i} = 1 \quad \forall C \in \mathcal{C}. \quad (9)$$

There are no separator variables, no per-family conditional variables and no global joint variable; the tree structure enters only through the equalities below.

**Marginalization.** Let  $\pi_{C \rightarrow S}$  map a state of  $C$  to its restriction to  $S \subseteq A(C)$ , and define the 0/1 aggregation matrix

$$M_{r,j}^{C \rightarrow S} = \mathbf{1}[\pi_{C \rightarrow S}(j) = r], \quad \text{so} \quad (M^{C \rightarrow S} q_C)[r] = (q_C \upharpoonright S)(r). \quad (10)$$

**Separator consistency.** For every edge with a non-empty separator, the two endpoints must agree on it:

$$\sum_{j: \pi_{C \rightarrow S}(j)=r} q_{C,j} = \sum_{k: \pi_{D \rightarrow S}(k)=r} q_{D,k} \quad \forall (C, D) \in E, \quad \forall r \in \{0, \dots, 2^{|S_{CD}|-1}\}, \quad (11)$$

equivalently  $q_C \upharpoonright S_{CD} = q_D \upharpoonright S_{CD}$ : linear equalities,  $2^{|S_{CD}|}$  per edge. No messages are computed; the consistency conditions are imposed simultaneously with all others, which is what distinguishes this formulation from message passing.

**Sentences.** A Type-1 sentence hosted on  $C_h \supseteq \text{atoms}(\varphi)$  contributes

$$\ell \leq \langle A_\varphi^{(C_h)}, q_{C_h} \rangle \leq u, \quad (12)$$

and a Type-2 sentence contributes the cleared pair

$$\ell \langle A_\psi^{(C_h)}, q_{C_h} \rangle \leq \langle A_{\varphi \wedge \psi}^{(C_h)}, q_{C_h} \rangle \leq u \langle A_\psi^{(C_h)}, q_{C_h} \rangle. \quad (13)$$

No division appears, and consistently with Remark 1 the rows (13) are vacuously satisfied when  $P(\psi) = 0$ : a Type-2 sentence constrains nothing on a null conditioning event.



**Independence assertions.** For a hosted  $(X \perp\!\!\!\perp Y \mid S)$  with  $X = \{x\}$ , one equality per configuration  $(\mathbf{y}, \mathbf{s})$ . With  $S \neq \emptyset$ , writing the four indicators for  $\{x=1, Y=\mathbf{y}, S=\mathbf{s}\}$ ,  $\{S=\mathbf{s}\}$ ,  $\{x=1, S=\mathbf{s}\}$ ,  $\{Y=\mathbf{y}, S=\mathbf{s}\}$  as  $A_a, A_b, A_c, A_d$ :

$$\langle A_a, q_{C_h} \rangle \langle A_b, q_{C_h} \rangle - \langle A_c, q_{C_h} \rangle \langle A_d, q_{C_h} \rangle = 0, \quad (14)$$

which is (2) verbatim; and with  $S = \emptyset$  the marginal form  $\langle A_a, q \rangle - \langle A_b, q \rangle \langle A_c, q \rangle = 0$  for  $P(x, \mathbf{y}) = P(x)P(\mathbf{y})$ . These rows are bilinear, hence nonconvex, and are the reason a global solver is required.

Two features of this encoding matter. First, only the  $x=1$  slice is needed: with  $(\mathbf{y}, \mathbf{s})$  fixed a binary  $X$  has one degree of freedom, so the  $x=0$  equality follows. Second, the configurations  $\mathbf{y}$  must range over *whole* assignments of  $Y$ . Replacing  $(X \perp\!\!\!\perp Y \mid S)$  by the pairwise family  $\{(X \perp\!\!\!\perp t \mid S)\}_{t \in Y}$  is strictly weaker for  $|Y| \geq 2$  — pairwise independence does not imply joint independence — and under-constrains the program, yielding spuriously loose bounds.

**Objective.** Let  $C_a$  be a cluster containing the query atom  $a$  and all evidence atoms (hypothesis H4). Without evidence the read-off is linear,

$$\rho_a(q) = \langle A_{a=1}^{(C_a)}, q_{C_a} \rangle = P(a=1). \quad (15)$$

With evidence  $e$  the ratio is reformulated with one auxiliary  $t \in [0, 1]$ ,

$$\rho_a(q) = t, \quad t \cdot \langle A_e^{(C_a)}, q_{C_a} \rangle = \langle A_{a=1}^{(C_a)} \cdot A_e^{(C_a)}, q_{C_a} \rangle, \quad (16)$$

so  $t = P(a=1 \mid e)$  whenever  $P(e) > 0$ . Since  $t$  is unconstrained where  $P(e) = 0$ , a global solver must be prevented from exploiting that degeneracy, and a denominator floor

$$\langle A_e^{(C_a)}, q_{C_a} \rangle \geq \varepsilon \quad (17)$$

is imposed for a small  $\varepsilon > 0$ . Remark 6 shows this floor is a soundness hypothesis, not a numerical convenience.

**The program.** Collecting (9)–(16),

$$\begin{aligned} \underline{P}(a), \bar{P}(a) &= \min / \max \quad \rho_a(q) \\ \text{s.t.} \quad &\sum_i q_{C,i} = 1, \quad q_{C,i} \in [0, 1] \quad \forall C \in \mathcal{C} \\ &q_C \upharpoonright S_{CD} = q_D \upharpoonright S_{CD} \quad \forall (C, D) \in E, S_{CD} \neq \emptyset \\ &\text{(12), (13)} \quad \forall \text{ hosted sentence} \\ &\text{(14)} \quad \forall \text{ hosted assertion.} \end{aligned} \quad (18)$$

## 5 A worked example

We work `examples/ktree_fr_k2_n4_example.lcn` completely. It is a  $k$ -tree with  $k = 2$  on four atoms — *loopy* after moralization, so the example is not a degenerate singly-connected case — yet small enough that every variable and every constraint can be written down.

## 5.1 The instance

$$\begin{aligned}
s_0 : \quad & 0.084382 \leq P(x_1) \leq 0.684382 \\
s_1 : \quad & 0.01 \leq P(\neg x_2 \mid \neg x_1) \leq 0.572656 \\
s_2 : \quad & 0.092785 \leq P(x_3 \mid x_1 \wedge x_2) \leq 0.692785 \\
s_3 : \quad & 0.068242 \leq P(x_0 \mid x_1 \wedge x_3) \leq 0.668242
\end{aligned}$$

The chain-graph families, in construction order, are

$$x_1 \mid \emptyset, \quad x_2 \mid x_1, \quad x_3 \mid x_1, x_2, \quad x_0 \mid x_1, x_3,$$

and the Local Markov Condition yields exactly one assertion,

$$(x_0 \perp\!\!\!\perp x_2 \mid x_1, x_3).$$

## 5.2 The junction tree

Min-fill elimination produces four clusters of at most three atoms, arranged in a path (Figure 1). In the convention (6) this is  $w + 1 = 3$ , i.e. treewidth  $w = 2 = k$ , as expected for a  $k$ -tree:

$$\underbrace{\{x_3\}}_{C_4} - \underbrace{\{x_2, x_3\}}_{C_3} - \underbrace{\{x_1, x_2, x_3\}}_{C_2} - \underbrace{\{x_1, x_3, x_0\}}_{C_1},$$

with separators  $S_{12} = \{x_1, x_3\}$ ,  $S_{23} = \{x_2, x_3\}$ ,  $S_{34} = \{x_3\}$ . Hosting assigns  $s_0, s_3$  to  $C_1$  and  $s_1, s_2$  to  $C_2$ .

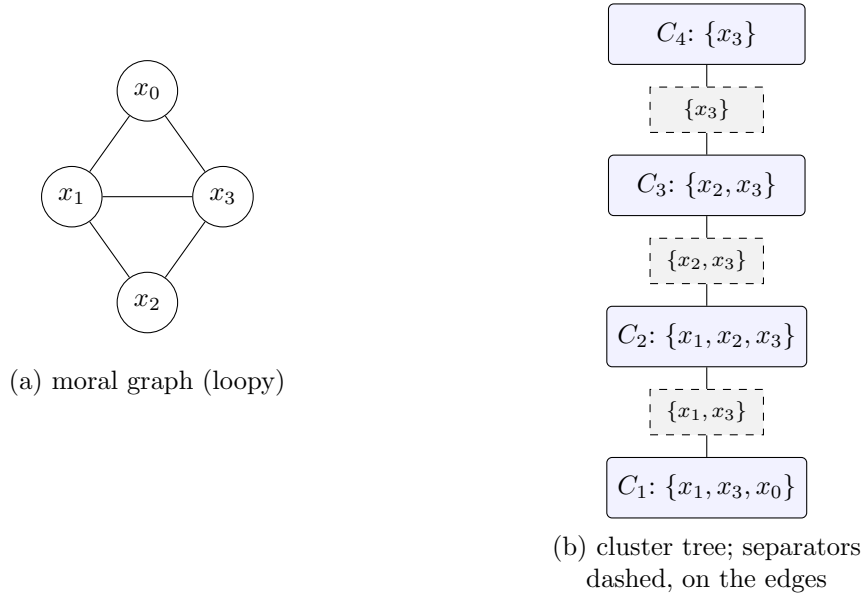


Figure 1: The example instance. Moralization makes  $\{x_1, x_2, x_3\}$  and  $\{x_0, x_1, x_3\}$  triangles, so the graph is loopy; the cluster tree nonetheless has width 2. The separator  $S_{12} = \{x_1, x_3\}$  equals the conditioning set of the sole assertion  $(x_0 \perp\!\!\!\perp x_2 \mid x_1, x_3)$ , which is therefore separator-entailed and never imposed.

### 5.3 Separator-entailment discharges the assertion

The chordal graph  $G^*$  of Definition 2 — two atoms adjacent iff they co-occur in a cluster — has edge set

$$x_0x_1, \quad x_0x_3, \quad x_1x_2, \quad x_1x_3, \quad x_2x_3,$$

and in particular  $x_0x_2 \notin G^*$ . Deleting  $S = \{x_1, x_3\}$  therefore disconnects  $x_0$  from  $x_2$ : the only paths between them run through  $x_1$  or  $x_3$ . So  $S$  separates  $x_0$  from  $x_2$  in  $G^*$ , and the sole assertion  $(x_0 \perp\!\!\!\perp x_2 \mid x_1, x_3)$  is separator-entailed — case 1 of the triage in Section 3.3. It is *not* imposed, and the constraint count below shows zero assertion rows. By Remark 4(a) the program is consequently a *linear* program, even though the instance is loopy.

This is the mechanism of Theorem 1 in its clearest form: the independence is enforced by the tree's *topology* — through Lemma 2's reconstruction, which produces a joint factorizing over the chordal graph and hence satisfying its global Markov property — rather than by any algebraic row.

### 5.4 The program in full

**Variables (22).**  $q_{C_1} \in \Delta(8)$  over  $(x_1, x_3, x_0)$ ;  $q_{C_2} \in \Delta(8)$  over  $(x_1, x_2, x_3)$ ;  $q_{C_3} \in \Delta(4)$  over  $(x_2, x_3)$ ;  $q_{C_4} \in \Delta(2)$  over  $(x_3)$ . Total  $8 + 8 + 4 + 2 = 22$ .

**Simplex (4).**  $\sum_i q_{C_k, i} = 1$  for  $k = 1, \dots, 4$ .

**Separator consistency (4 + 4 + 2 = 10).** Writing  $q_C[\cdot]$  for a cell of cluster  $C$ , edge  $C_1 - C_2$  over  $\{x_1, x_3\}$  gives four equalities,

$$\begin{aligned} q_{C_1}[x_1=0, x_3=0, x_0=0] + q_{C_1}[x_1=0, x_3=0, x_0=1] &= q_{C_2}[x_1=0, x_2=0, x_3=0] + q_{C_2}[x_1=0, x_2=1, x_3=0], \\ q_{C_1}[x_1=0, x_3=1, x_0=0] + q_{C_1}[x_1=0, x_3=1, x_0=1] &= q_{C_2}[x_1=0, x_2=0, x_3=1] + q_{C_2}[x_1=0, x_2=1, x_3=1], \\ q_{C_1}[x_1=1, x_3=0, x_0=0] + q_{C_1}[x_1=1, x_3=0, x_0=1] &= q_{C_2}[x_1=1, x_2=0, x_3=0] + q_{C_2}[x_1=1, x_2=1, x_3=0], \\ q_{C_1}[x_1=1, x_3=1, x_0=0] + q_{C_1}[x_1=1, x_3=1, x_0=1] &= q_{C_2}[x_1=1, x_2=0, x_3=1] + q_{C_2}[x_1=1, x_2=1, x_3=1]; \end{aligned}$$

edge  $C_2 - C_3$  over  $\{x_2, x_3\}$  gives four more,

$$q_{C_2}[x_1=0, x_2=\beta, x_3=\gamma] + q_{C_2}[x_1=1, x_2=\beta, x_3=\gamma] = q_{C_3}[x_2=\beta, x_3=\gamma], \quad \beta, \gamma \in \{0, 1\};$$

and edge  $C_3 - C_4$  over  $\{x_3\}$  gives two,

$$q_{C_3}[x_2=0, x_3=\gamma] + q_{C_3}[x_2=1, x_3=\gamma] = q_{C_4}[x_3=\gamma], \quad \gamma \in \{0, 1\}.$$

**Sentence rows (8).** On  $C_1$ , from  $s_0$  (Type-1) and  $s_3$  (Type-2, cleared per (13)):

$$\begin{aligned} 0.084382 &\leq \langle A_{x_1}^{(C_1)}, q_{C_1} \rangle \leq 0.684382, \\ 0.068242 \langle A_{x_1 \wedge x_3}^{(C_1)}, q_{C_1} \rangle &\leq \langle A_{x_0 \wedge x_1 \wedge x_3}^{(C_1)}, q_{C_1} \rangle \leq 0.668242 \langle A_{x_1 \wedge x_3}^{(C_1)}, q_{C_1} \rangle. \end{aligned}$$

On  $C_2$ , from  $s_1$  and  $s_2$  (both Type-2):

$$\begin{aligned} 0.01 \langle A_{\neg x_1}^{(C_2)}, q_{C_2} \rangle &\leq \langle A_{\neg x_2 \wedge \neg x_1}^{(C_2)}, q_{C_2} \rangle \leq 0.572656 \langle A_{\neg x_1}^{(C_2)}, q_{C_2} \rangle, \\ 0.092785 \langle A_{x_1 \wedge x_2}^{(C_2)}, q_{C_2} \rangle &\leq \langle A_{x_3 \wedge x_1 \wedge x_2}^{(C_2)}, q_{C_2} \rangle \leq 0.692785 \langle A_{x_1 \wedge x_2}^{(C_2)}, q_{C_2} \rangle. \end{aligned}$$

**Independence rows (0).** None, by separator-entailment as argued above.

**Totals.** 22 variables and 22 constraints (4 simplex, 10 separator, 8 sentence, 0 LMC), against a joint formulation’s  $2^4 = 16$  variables. The example is deliberately small enough that the treewidth formulation is not yet a saving; the saving is asymptotic,  $O(n2^{w+1})$  against  $2^n$ , and Table 2 shows the same machinery on two five-atom instances.

## 5.5 Results

Solving (18) for each atom with SCIP (global, gap 0) and comparing against the independent  $2^n$  global engine:

| engine                           | $P(x_0)$ | $P(x_1)$             | $P(x_2)$             | $P(x_3)$ |
|----------------------------------|----------|----------------------|----------------------|----------|
| direct $2^n$ formulation, global | [0, 1]   | [0.084382, 0.684382] | [0.134877, 0.996844] | [0, 1]   |
| CREDALJT (D5, SCIP)              | [0, 1]   | [0.084382, 0.684382] | [0.134877, 0.996844] | [0, 1]   |

The two engines agree on every atom to every digit reported, with all eight of the reference optimizations certified at optimality gap 0. CREDALJT reports `d5_exact` true, maximum cluster 3 atoms, and a total time of about 1.8 s.

The vacuous  $[0, 1]$  on  $x_0$  and  $x_3$  is genuine rather than a solver artifact, and the independent oracle confirms it: no sentence constrains  $P(x_3)$  except through conditioning events that can themselves be driven to probability zero, at which point (13) becomes vacuous (Remark 1) and frees the atom. Setting  $P(x_1 \wedge x_3) = 0$  likewise releases  $s_3$  and hence  $x_0$ . Both endpoints are thus attained by feasible witnesses.

The separator marginals attained at an optimum make the mechanism concrete. Table 1 lists them at the maximizer of  $P(x_1)$ . Both endpoints of every edge agree on these values by (11); this is the calibrated family of cluster marginals that Lemma 2 reassembles into a joint of  $\mathcal{D}_{\text{LCN}}$  attaining the bound.

Table 1: Separator marginals  $q_C \upharpoonright S$  at the maximizer of  $P(x_1)$ , listed over the assignments of  $S$ . Both clusters incident to an edge realize the same vector, by (11).

| edge        | separator $S$  | $q \upharpoonright S$ by assignment of $S$ |            |            |            |
|-------------|----------------|--------------------------------------------|------------|------------|------------|
| $C_1 - C_2$ | $\{x_1, x_3\}$ | 00: 0.3125                                 | 01: 0.0032 | 10: 0.6844 | 11: 0.0000 |
| $C_2 - C_3$ | $\{x_2, x_3\}$ | 00: 0.6844                                 | 01: 0.0032 | 10: 0.3125 | 11: 0.0000 |
| $C_3 - C_4$ | $\{x_3\}$      | 0: 0.9968                                  | 1: 0.0032  |            |            |

## 5.6 Two contrasting structures

The running example imposes no assertion row. Table 2 reports two five-atom instances that do, so that the bilinear rows are exercised rather than discharged; one of them also exhibits a compound node.

For **asia** the tree is the path  $\{X\} - \{D, X\} - \{C, D, X\} - \{B, C, D\} - \{B, S, C\}$  with separators  $\{X\}, \{D, X\}, \{C, D\}, \{B, C\}$ , so the separator count is  $2 + 4 + 4 + 4 = 14$ , as (11) predicts. For **alarm** the tree branches:  $\{E\} - \{E, A\}$  with children  $\{B, E, A\}$  and  $\{A, C, D\}$ , giving  $2 + 4 + 2 = 8$ .

## 6 Exactness

This section states and proves the guarantees. The development is closed: it uses only Lemma 1 from Section 2 and Lemma 2 below, and in particular needs no notion of realizability, no property

Table 2: Program size on two further instances, measured. **asia** has a hosted *conditional* assertion  $(C \perp\!\!\!\perp B \mid S)$  contributing  $2^{|Y|}2^{|S|} = 4$  bilinear rows; **alarm** has a hosted *marginal* assertion  $(B \perp\!\!\!\perp E)$  with  $S = \emptyset$  contributing  $2^{|Y|} = 2$  rows, and its undirected component becomes the compound node C-D whose cluster carries atoms  $\{A, C, D\}$ .

| instance           | clus. | max $ A(C) $ | edges | vars | constr. | simplex | sep. | sent. | LMC |
|--------------------|-------|--------------|-------|------|---------|---------|------|-------|-----|
| examples/asia.lcn  | 5     | 3            | 4     | 30   | 33      | 5       | 14   | 10    | 4   |
| examples/alarm.lcn | 4     | 3            | 3     | 22   | 24      | 4       | 8    | 10    | 2   |

of the strong extension, and no result about any other engine.

**Definition 2** (Cluster-feasible set, hosting, separator-entailment, read-off). Let  $T$  be a clique tree over clusters  $\mathcal{C}$ , built by the standard moralize  $\rightarrow$  triangulate  $\rightarrow$  clique-tree pipeline, and let  $G^*$  be the chordal graph whose maximal cliques are the clusters (two atoms adjacent iff they co-occur in a cluster).

- The *cluster-feasible set* is

$$\mathcal{G} = \{(q_C)_{C \in \mathcal{C}} : q_C \in \Delta(2^{|C|}) \text{ for each } C; \\ q_C \upharpoonright S = q_D \upharpoonright S \text{ for every edge } (C, D) \in T, S = C \cap D; \\ \text{every hosted constraint holds on its host cluster}\}.$$

- A constraint  $c$  is *hosted* if some cluster  $C_h \supseteq \text{scope}(c)$  exists; its row is then imposed on  $q_{C_h}$ 's marginal.
- An LMC assertion  $(X \perp\!\!\!\perp Y \mid S)$  is *separator-entailed* if  $S$  separates  $X$  from  $Y$  in  $G^*$ .
- For a query atom  $a$  and evidence  $e$  lying together in a cluster  $C_a$ , the *read-off* is  $\rho_a(q) = \sum_{i:a=1} q_{C_a}[i]$  (no evidence), or  $\rho_a(q) = (\sum_{i:a=1,e} q_{C_a}[i]) / (\sum_{i:e} q_{C_a}[i])$  (with evidence).
- $\pi(p) = (p \upharpoonright C)_{C \in \mathcal{C}}$  is the *projection* of a joint onto the clusters.

The reconstruction direction needs the following lemma, whose usual statement contains two defects we must repair: it is normally asserted with a *uniqueness* claim that is false in the presence of zero separator cells, and the division in the product formula is left unaddressed.

**Lemma 2** (RIP reconstruction, with zeros). *Let  $T$  be a clique tree with the running-intersection property over clusters  $\mathcal{C}$ , and let  $(q_C) \in \prod_C \Delta(2^{|C|})$  be separator-consistent:  $q_C \upharpoonright S = q_D \upharpoonright S$  for every edge  $(C, D)$  with  $S = C \cap D$ . Then there exists a joint distribution  $p \in \Delta(2^{\mathbf{V}})$  with  $p \upharpoonright C = q_C$  for every  $C \in \mathcal{C}$ , and  $p$  factorizes over the cliques of  $G^*$ . Such  $p$  is unique if every separator marginal is strictly positive, but not in general.*

*Proof.* Induction on  $|\mathcal{C}|$ . If  $|\mathcal{C}| = 1$  take  $p = q_C$ .

Let  $|\mathcal{C}| \geq 2$  and pick a leaf cluster  $C$  of  $T$  with neighbour  $D$  and separator  $S = C \cap D$ . Let  $T' = T - C$ ; by the running-intersection property  $C \setminus S$  meets no cluster of  $T'$ , and  $(q_{C'})_{C' \in T'}$  is still separator-consistent, so the inductive hypothesis gives a joint  $p'$  over  $\mathbf{V}' = \bigcup_{C' \in T'} C'$  with  $p' \upharpoonright C' = q_{C'}$  for all  $C' \in T'$ . Note  $p' \upharpoonright S = (p' \upharpoonright D) \upharpoonright S = q_D \upharpoonright S = q_C \upharpoonright S =: q_S$ , using separator consistency.

Define a conditional kernel on the fresh atoms  $C \setminus S$ : for each assignment  $s$  of  $S$ ,

$$\kappa(\cdot \mid s) = \begin{cases} q_C(\cdot, s)/q_S(s), & q_S(s) > 0, \\ \delta_{c_0}, & q_S(s) = 0, \end{cases}$$

where  $c_0$  is any fixed assignment of  $C \setminus S$ . This is well defined: when  $q_S(s) > 0$  the right-hand side is non-negative and sums over  $C \setminus S$  to  $q_S(s)/q_S(s) = 1$ , so it is a distribution. Now set  $p(x) = p'(x_{\mathbf{V}'}) \cdot \kappa(x_{C \setminus S} \mid x_S)$ .

$p$  is a distribution: it is non-negative, and summing over  $x_{C \setminus S}$  first gives  $p'$ , which sums to 1.

*The arbitrary choice is irrelevant.* On  $\{x : q_S(x_S) = 0\}$  we have  $p'(x_{\mathbf{V}'}) = 0$ , because  $p' \upharpoonright S = q_S$  assigns zero mass to that event and  $p' \geq 0$ . Hence  $p$  vanishes there regardless of  $\kappa$ , so  $p$  does not depend on  $c_0$ . (This is also the precise sense in which the familiar quotient formula  $p = \prod_C q_C / \prod_{(C,D)} q_S$  is legitimate: a zero separator cell forces every contributing cluster cell to zero, since  $q_S = q_C \upharpoonright S$  is a sum of non-negative  $q_C$  entries, so numerator and denominator vanish together and the term is assigned 0.)

*Cluster marginals.* For  $C' \in T'$ : summing  $p$  over  $x_{C \setminus S}$  gives  $p'$ , so  $p \upharpoonright C' = p' \upharpoonright C' = q_{C'}$ . For  $C$  itself:  $p \upharpoonright C(c, s) = (\sum \text{of } p \text{ over } \mathbf{V}' \setminus S \text{ at fixed } x_S = s) \cdot \kappa(c \mid s) = p' \upharpoonright S(s) \kappa(c \mid s) = q_S(s) \kappa(c \mid s)$ , which equals  $q_C(c, s)$  when  $q_S(s) > 0$  and equals  $0 = q_C(c, s)$  when  $q_S(s) = 0$ . So  $p \upharpoonright C = q_C$ .

*Factorization.* Unwinding the induction,  $p$  is a product of one non-negative factor per cluster (namely  $q_C$  or the kernel  $\kappa$  derived from it, each a function of the atoms of that cluster only), hence factorizes over the cliques of  $G^*$ .

*Non-uniqueness.* If some separator cell  $q_S(s) = 0$ , then  $\kappa(\cdot \mid s)$  was chosen arbitrarily; any other choice yields a joint with the same cluster marginals, since all such joints vanish on that event. Hence  $p$  is not unique. When all separator marginals are positive the kernels are forced and the induction determines  $p$  uniquely.  $\square$

**Theorem 1** (CREDALJT exactness). *Suppose (H1)  $T$  has the running-intersection property (true by construction); (H2) every LCN sentence is hosted; (H3) every LMC assertion is either hosted or separator-entailed in  $G^*$ ; (H4) the query atom and all evidence atoms lie together in some cluster. Then for every such atom  $a$ ,*

$$\min_{q \in \mathcal{G}} \rho_a(q) = \min_{p \in \mathcal{D}_{\text{LCN}}} \rho_a(\pi(p)) = \underline{P}(a), \quad \max_{q \in \mathcal{G}} \rho_a(q) = \max_{p \in \mathcal{D}_{\text{LCN}}} \rho_a(\pi(p)) = \overline{P}(a),$$

*and likewise for a conditional query  $P(a \mid e)$  subject to Remark 6. Moreover the projection half holds without (H2)/(H3), so CREDALJT is always a valid outer bound.*

*Proof.* First note the read-off identity: for any joint  $p$ ,  $\rho_a(\pi(p)) = P_p(a)$  (resp.  $P_p(a \mid e)$ ), because  $\rho_a$  sums the cells of  $\pi(p)_{C_a} = p \upharpoonright C_a$  in which  $a$  (and  $e$ ) hold, which by definition of a marginal equals  $P_p(a)$  (resp. the ratio). So both sides optimize the *same* functional.

*Projection  $\mathcal{D}_{\text{LCN}} \rightarrow \mathcal{G}$  (needs only H1, H4).* Let  $p \in \mathcal{D}_{\text{LCN}}$  and set  $q_C := p \upharpoonright C$ , i.e.  $q = \pi(p)$ . Each  $q_C$  is a distribution (a marginal of one). Each separator equality holds because both  $q_C \upharpoonright S$  and  $q_D \upharpoonright S$  equal  $p \upharpoonright S$ . Each hosted constraint  $c$  holds: by Lemma 1 its value depends on  $p$  only through  $p \upharpoonright \text{scope}(c)$ , and since  $\text{scope}(c) \subseteq C_h$  we have  $q_{C_h} \upharpoonright \text{scope}(c) = p \upharpoonright \text{scope}(c)$ , so  $c$  holds on the host exactly because it holds for  $p$ . Hence  $q \in \mathcal{G}$ , with  $\rho_a(q) = P_p(a)$ . Therefore every value achievable over  $\mathcal{D}_{\text{LCN}}$  is achievable over  $\mathcal{G}$ , giving  $\min_{\mathcal{G}} \rho_a \leq \min_{\mathcal{D}_{\text{LCN}}} \rho_a$  and  $\max_{\mathcal{G}} \rho_a \geq \max_{\mathcal{D}_{\text{LCN}}} \rho_a$ : the cluster interval *contains* the true interval. This is the outer-bound claim, and it used neither H2 nor H3.

*Reconstruction  $\mathcal{G} \rightarrow \mathcal{D}_{\text{LCN}}$  (needs H2, H3).* Let  $q \in \mathcal{G}$ . By H1 and Lemma 2 there is a joint  $p$  with  $p \upharpoonright C = q_C$  for every cluster, factorizing over the cliques of  $G^*$ . We check  $p \in \mathcal{D}_{\text{LCN}}$ .

Sub-scope consistency: if  $A \subseteq C$  for some cluster  $C$ , then  $p \upharpoonright A = (p \upharpoonright C) \upharpoonright A = q_C \upharpoonright A$ , since marginalizing in stages agrees with marginalizing at once.

*Hosted constraints* (H2 and the hosted part of H3): let  $c$  be hosted on  $C_h$ . By the previous paragraph  $p \upharpoonright \text{scope}(c) = q_{C_h} \upharpoonright \text{scope}(c)$ , and  $q$  satisfies  $c$  on  $C_h$  by definition of  $\mathcal{G}$ ; by Lemma 1  $c$  therefore holds for  $p$ .

*Separator-entailed LMC* (the other part of H3): let  $(X \perp\!\!\!\perp Y \mid S)$  be separator-entailed, so  $S$  separates  $X$  from  $Y$  in  $G^*$ . Since  $p$  factorizes over the cliques of  $G^*$ , it satisfies the *global Markov property* of  $G^*$ : factorization implies global Markov for any undirected graph, and crucially this direction requires *no* positivity assumption (Lauritzen, *Graphical Models*, Prop. 3.8; only the converse, global Markov  $\Rightarrow$  factorization, needs  $p > 0$  via Hammersley–Clifford). Global Markov states that if  $S$  separates  $A$  from  $B$  in  $G^*$  then  $A \perp\!\!\!\perp B \mid S$  under  $p$ ; applying it to the connected components separated by  $S$  and then restricting to the subsets  $X$  and  $Y$  of those components (conditional independence is preserved under taking subsets, by the decomposition property of conditional independence) gives  $X \perp\!\!\!\perp Y \mid S$  under  $p$ , i.e. (2) holds. Note this assertion was never imposed on  $\mathcal{G}$ : it holds automatically for the reconstructed  $p$ .

Hence  $p \in \mathcal{D}_{\text{LCN}}$ , and  $\rho_a(q) = \rho_a(\pi(p)) = P_p(a)$  because  $\pi(p) = q$  on the host cluster. So every value achievable over  $\mathcal{G}$  is achievable over  $\mathcal{D}_{\text{LCN}}$ , giving the reverse inequalities. Combined with the projection half, the optima coincide. For a conditional query, by H4 the read-off is a functional of the single cluster  $C_a$  and both maps preserve  $q_{C_a}$ , so the same argument applies verbatim.  $\square$

**Remark 5** (Reading the proof). (a) The argument never mentions chains: exactness is a property of the *junction tree*, via RIP, hosting, separator-entailment and locality. It is in particular *not* a statement about singly-connected topologies — the worked example of Section 5 is loopy and exact. (b) The projection half is unconditional, so CREDALJT is *never unsound*: its interval always contains the truth, and H2/H3 decide only tightness. (c) H3 is *sufficient, not necessary*: on `examples/asia.lcn` and `examples/polytree.lcn` some assertions are neither hosted nor separator-entailed, yet CREDALJT is empirically exact — the untailed assertions simply do not bind. Where H3 fails *and* binds, the result is a valid but loose outer bound: on `examples/smokers.lcn` the six large-scope LMCs of the undirected component  $F_1 F_2 F_3$  are unhostable and untailed (within the width-6 cluster  $\{F_1, F_2, F_3, S_1, S_2, S_3\}$  no separator divides  $S_3$  from  $F_1$ ), and CREDALJT returns  $P(F_1) = [0, 1]$ .

**Remark 6** (The denominator floor is a soundness hypothesis). The conditional read-off is linearized by the auxiliary equality (16) together with the denominator floor (17). The floor is an extra constraint on  $\mathcal{G}$  that  $\mathcal{D}_{\text{LCN}}$  does not have, so the projection half of Theorem 1 — and with it the outer-bound guarantee — holds only for joints with  $P(e) \geq \varepsilon$ . If  $\min_{p \in \mathcal{D}_{\text{LCN}}} P(e) < \varepsilon$ , the returned conditional interval can be *unsound*. This is a soundness caveat, not merely a tightness one, and it is the price of admitting conditional queries into a globally solved formulation.

**Corollary 1** (Exactness on `ktree-fr` at treewidth cost). *On every `ktree-fr` instance, H1–H4 hold, so CREDALJT returns the exact  $\mathcal{D}_{\text{LCN}}$  marginal bounds at cost  $O(n 2^{k+1})$ .*

*Proof.* A  $k$ -tree is chordal, and in the `ktree-fr` class it is oriented as a DAG in construction order, every atom conditioning on the *full conjunction* of its  $k$  clique-parents. Those parents are pairwise adjacent in the  $k$ -tree, so moralization adds no edge and triangulation is a no-op:  $G^*$  is the  $k$ -tree itself and its maximal cliques — the clusters — are exactly the  $(k+1)$ -cliques, giving H1 by construction. Every sentence is either a root marginal or scoped to a child with its  $\leq k$  parents, which co-occur in a cluster: H2. Each LMC assertion produced by the Local Markov Condition has as its conditioning set a full clique-separator of the  $k$ -tree, which separates the two sides in

$G^*$ , so each is hosted or separator-entailed: H3. Any single query atom lies in a  $(k+1)$ -clique: H4. Theorem 1 applies, and the cost is  $\sum_C 2^{|C|} = O(n) \cdot 2^{k+1}$ , over  $O(n)$  clusters each of size  $k+1$ .  $\square$

**Remark 7** (What the `ktree-fr` corollary does and does not claim). Corollary 1 is a statement about *cost*: exactness at  $O(n2^{k+1})$  rather than  $2^n$ . It should *not* be read as “the one loopy topology where a treewidth-bounded engine beats the strong-extension engines on accuracy”. On the loopy realizable  $k$ -tree of Section 5 Credal VE is also exact, atom for atom. On a larger  $k=3$  instance (`ktree_fr_k3_n5.1.lcn`) Credal VE fails not by being loose but by not finishing: the credal network there is tiny (512 vertex combinations across five families), yet elimination did not terminate within 200s, stalling inside potential pruning, while CREDALJT solved the instance in about a second. The separation between CREDALJT and Credal VE on this family is therefore one of *tractability*, not of accuracy. Loopiness alone is not a reason to expect looseness.

**Example 1** (Verified end to end). Section 5 is the worked case at  $k = 2$ . Independently: on `ktree_fr_k3_n5.1.lcn` the two maximal cliques are  $\{x_0, x_1, x_3, x_4\}$  and  $\{x_1, x_2, x_3, x_4\}$  (treewidth 3), the sole LMC assertion  $(x_0 \perp\!\!\!\perp x_2 \mid x_1, x_3, x_4)$  has conditioning set equal to the separator between them, and CREDALJT reproduces the certified global  $2^n$  bounds on all five atoms; on `examples/chain.lcn` the clusters are the 7 edge-pairs (treewidth 1: 28 program variables against a  $2^8 = 256$  joint); and on `examples/polytree.lcn` the collider cliques  $\{x_0, x_1, x_2\}$  and  $\{x_3, x_4, x_5\}$  give treewidth 2.

## 6.1 Position among the engines

Table 3 places CREDALJT against the other shipped engines, and Table 4 reports the observed bound character by topology.

Table 3: Bound character per engine. “Targets” is the feasible set the engine optimizes; “character” is per Definition 1 *against that set*. The fourth column gives the condition for the returned interval to equal the  $\mathcal{D}_{\text{LCN}}$  truth.

| Engine                      | Targets                    | Character (vs. target)        | Exact w.r.t. $\mathcal{D}_{\text{LCN}}$ when | Cost             |
|-----------------------------|----------------------------|-------------------------------|----------------------------------------------|------------------|
| Direct $2^n$ , global       | $\mathcal{D}_{\text{LCN}}$ | exact if certified            | endpoint certified                           | $2^n$ , global   |
| Direct $2^n$ , local        | $\mathcal{D}_{\text{LCN}}$ | unreliable                    | — (may over-tighten)                         | $2^n$ , ipopt    |
| <b>CredalJT</b> (D5)        | $\mathcal{D}_{\text{LCN}}$ | outer always; exact if H1–H4  | H1–H4 (Thm. 1)                               | $O(n2^{w+1})$    |
| Credal VE                   | $\mathcal{D}_{\text{SE}}$  | exact (unconditional queries) | no Gap A and no Gap B                        | per-target elim. |
| Credal CTE ( $\epsilon=0$ ) | $\mathcal{D}_{\text{SE}}$  | <b>inner</b>                  | clean s.c.; no cross-family sent.            | two-pass tree    |
| ApproxLP                    | $\mathcal{D}_{\text{SE}}$  | <i>inner</i>                  | no gaps + no stall                           | coord. descent   |
| Interval BP (interval)      | $\mathcal{D}_{\text{SE}}$  | outer; exact on tree          | tree + no gaps                               | loopy messages   |
| Credal IJGP                 | $\mathcal{D}_{\text{SE}}$  | outer; exact if $i \geq w$    | $i \geq w$ + no gaps                         | $2^i$ /cluster   |
| ARIEL                       | $\mathcal{D}_{\text{LCN}}$ | outer always                  | see the ARIEL note                           | factor messages  |

\* On **alarm** the global backend leaves several endpoints uncertified and the *local* variant over-tightens; the certified values come from the dedicated verifiers.

In summary: on bounded-treewidth instances CREDALJT is exact under H1–H4 and a valid outer bound otherwise, at  $O(n2^{w+1})$  rather than  $2^n$ ; it remains applicable on instances where the direct formulation is solvable but its global solver certifies no endpoint. For conditional queries Remark 6 applies.



Table 4: Bound character by topology, unconditional marginals. “ex” = exact w.r.t.  $\mathcal{D}_{\text{LCN}}$ , “out” = valid outer bound, “in” = inner (unsound as a guarantee), “n/f” = did not finish.

| Topology                                   | Exact-glob | CREDALJT   | CVE        | CTE       | ApproxLP | IBP-int |
|--------------------------------------------|------------|------------|------------|-----------|----------|---------|
| Markov chain, tree, polytree (realizable)  | ex         | ex         | ex         | ex        | ex       | ex      |
| Chain graph with blocks ( <b>alarm</b> )   | ex*        | ex         | ex         | ex        | ex       | ex      |
| Cross-family sentence ( <b>d4_biting</b> ) | ex         | ex         | ex         | <b>in</b> | ex       | ex      |
| Non-realizable Type-1                      | ex         | ex         | out        | out       | out      | out     |
| Non-realizable Type-2                      | ex         | ex         | out        | out       | out      | out     |
| Loopy $k$ -tree, realizable, $k=2$ (§5)    | ex         | ex         | <b>ex</b>  | —         | —        | —       |
| Loopy $k$ -tree, realizable, $k=3$         | ex         | ex         | <b>n/f</b> | —         | —        | —       |
| Undirected block ( <b>smokers</b> )        | ex         | <b>out</b> | out        | out       | in       | out     |

## 7 A self-contained subsection for a paper

### Exact inference at treewidth cost

Let an LCN over binary atoms  $\mathbf{V}$  consist of sentences bounding  $P(\varphi)$  or  $P(\varphi \mid \psi)$ , read in cleared form  $\ell P(\psi) \leq P(\varphi \wedge \psi) \leq u P(\psi)$ , together with the independence assertions  $(X \perp\!\!\!\perp Y \mid S)$  derived by the Local Markov Condition, imposed as  $P(\mathbf{x}, \mathbf{y}, \mathbf{s})P(\mathbf{s}) = P(\mathbf{x}, \mathbf{s})P(\mathbf{y}, \mathbf{s})$ . These determine a credal set  $\mathcal{D}_{\text{LCN}} \subseteq \Delta(2^{\mathbf{V}})$ , and inference seeks  $P(a) = \min_{\mathcal{D}_{\text{LCN}}} P(a)$  and  $\bar{P}(a) = \max_{\mathcal{D}_{\text{LCN}}} P(a)$ . Optimizing over  $\mathcal{D}_{\text{LCN}}$  directly costs  $2^n$  variables; compiling to a credal network and propagating is cheaper but optimizes a strictly larger set, the strong extension.

CREDALJT avoids both. Moralize the chain graph induced by the program’s families, triangulate, and build a clique tree  $T$  over clusters  $\mathcal{C}$  with the running-intersection property; let  $G^*$  be the associated chordal graph. Give each cluster a variable vector  $q_C \in \Delta(2^{|C|})$  over the joint of its atoms and solve

$$\begin{aligned}
& \min / \max \quad \rho_a(q) \\
& \text{s.t.} \quad \sum_i q_{C,i} = 1, \quad q_{C,i} \geq 0 & \forall C \in \mathcal{C}, \\
& \quad q_C \upharpoonright S = q_D \upharpoonright S & \forall (C, D) \in T, \quad S = C \cap D, \\
& \quad \ell \langle A_\psi, q_{C_h} \rangle \leq \langle A_{\varphi \wedge \psi}, q_{C_h} \rangle \leq u \langle A_\psi, q_{C_h} \rangle & \forall \text{ sentence hosted on } C_h, \\
& \quad \langle A_a, q \rangle \langle A_b, q \rangle = \langle A_c, q \rangle \langle A_d, q \rangle & \forall \text{ assertion hosted,}
\end{aligned}$$

where  $A_\varphi$  is the 0/1 indicator of  $\varphi$  over the host cluster’s states, a constraint is *hosted* on any cluster containing its scope, and  $\rho_a$  reads  $P(a)$  (or  $P(a \mid e)$ , via one auxiliary variable) off a cluster containing  $a$  and the evidence. An assertion whose conditioning set separates its two sides in  $G^*$  is *separator-entailed* and is omitted: the tree’s topology already enforces it. The program is nonconvex through the bilinear assertion rows and is solved globally.

**Theorem 2.** *Assume (H1)  $T$  has the running-intersection property; (H2) every sentence is hosted; (H3) every assertion is hosted or separator-entailed in  $G^*$ ; (H4) the query and evidence atoms share a cluster. Then the program’s optima equal  $\underline{P}(a)$  and  $\bar{P}(a)$  exactly. Moreover the direction giving containment of the true interval requires only H1 and H4, so the returned interval is always a valid outer bound.*

The proof has two halves. Projection: for  $p \in \mathcal{D}_{\text{LCN}}$ , the cluster marginals  $p \upharpoonright C$  are separator-consistent and satisfy every hosted row, because a constraint’s value depends on  $p$  only through its scope, which lies inside the host; hence every value attained on  $\mathcal{D}_{\text{LCN}}$  is attained on the relaxation, which is the outer-bound claim. Reconstruction: a separator-consistent family of cluster marginals

reassembles, by running intersection, into a joint  $p$  with those marginals which factorizes over the cliques of  $G^*$  (the reassembly needs care where separator cells vanish: the conditional kernels are then arbitrary but immaterial, since  $p$  vanishes there regardless). Hosted rows transfer back to  $p$  by locality, and separator-entailed assertions hold automatically, since factorization implies the global Markov property of  $G^*$  with no positivity assumption. Full proofs are in the appendix.

The cost is  $\sum_C 2^{|C|} = O(n2^{w+1})$  variables in the treewidth  $w$ , against  $2^n$  for the direct formulation, plus  $O(n2^w)$  separator equalities. The constraint system is query-independent, so all marginals are obtained from one program by swapping the objective. As a consequence, on a  $k$ -tree rendered so that each atom conditions on the full conjunction of its  $k$  clique-parents, moralization and triangulation are no-ops, every hypothesis holds, and exact bounds are obtained at  $O(n2^{k+1})$ . Empirically, on a loopy four-atom  $k$ -tree the formulation reproduces a certified global  $2^n$  reference on every atom, using 22 variables and 22 constraints.

## 8 Experimental setup

All reported bounds were computed on the stated instances with the cluster program of Section 4, solved by a spatial branch-and-bound global solver at gap tolerance 0, and independently cross-checked against the direct  $2^n$  formulation over  $\mathcal{D}_{\text{LCN}}$  under the same global solver. On the instance of Section 5 the reference reported status *confirmed* at optimality gap 0 for all eight endpoint optimizations, and the two formulations agreed on every atom to the precision displayed.

The junction trees, separator equalities and constraint counts of Section 5 and Table 2 were read off the constructed cluster tree and constraint system directly rather than reconstructed by hand. All instances are shipped with the library: the running example is a  $k$ -tree with  $k = 2$  on four atoms, generated in the family-realizable regime, and the two instances of Table 2 are the standard **asia** and **alarm** programs.

## 9 Provenance

Every interval and every program-size figure in this document was computed as described in Section 8. A value attributed to the global  $2^n$  reference is reported as exact only where that optimization was certified at gap 0.

Two classes of statement are cited rather than re-derived here. The **smokers** and **asia/polytree** observations of Remark 5, the **alarm** certification caveat under Table 4, and the comparative rows of Tables 3 and 4 are taken from the consolidated reference **docs/properties.tex**, which reports them with its own provenance. The non-termination of Credal VE on the  $k=3$  instance in Remark 7 is likewise cited from that document.

Related notes in **docs/**: **properties.tex** consolidates the exactness results for all engines and is the source of the theory reproduced in Section 6; **cjt.exactness.tex** is a forensic companion investigating a reported CREDALJT-versus-oracle discrepancy on **ktree\_fr\_k3\_n5\_1.1cn** that does not reproduce; **tighter\_approximation.tex** defines the D1–D5 scheme taxonomy in which CREDALJT is D5; and **lrs.tex** covers the vertex enumeration that CREDALJT, uniquely among the credal-network engines, does not need.